Personalized AI Research Agent — Production-Grade Project Plan

> **Goal:** Build a personalized AI research agent that ingests **any** document (PDF, Word, Markdown, HTML, arXiv, and more), indexes it, and answers questions with cited, grounded, streaming answers over REST + chat interfaces.
>
> **Learning outcomes:** deep hands-on knowledge of **AI/LLMs & RAG**, **REST API design**, and **databases** (relational + search + vector + cache), plus the ops skills (Docker, CI/CD, observability) that separate a "college project" from a production system.

---

1. Architecture Decision — What to Build and Why

Recommendation: Modular Monolith (single deployable, strict internal module boundaries)

You have three realistic choices:

Pattern	What it is	Fit for you
Monolithic	One codebase, one deployable, no internal boundaries	✕ Fast to start but rots into spaghetti; you won't learn clean design

Microservices	Many independently deployed services	✗ Premature. Network calls, distributed tracing, service discovery, and 6 databases to babysit — huge ops tax for a solo builder. You'd spend your time on plumbing, not learning AI
Modular Monolith ✓	**One deployable**, but code is split into **strictly isolated modules** that talk only through defined interfaces	✓ **The sweet spot.** Production-grade discipline, microservice-ready boundaries, but one thing to deploy and debug

Why the modular monolith wins for this project:

1. **You learn clean architecture without distributed-systems pain.** Modules communicate via interfaces/events, not HTTP — but the boundaries are real, enforced, and reviewable.
2. **It's the industry default for new products** (Shopify, GitHub, and most startups run modular monoliths, not microservices).
3. **Any module can later be extracted into its own service** with minimal rework — e.g., if the ingestion pipeline needs to scale independently, you lift it out.
4. **One `docker compose up` runs the whole thing** — great for a portfolio/demo.

Module boundaries (each is its own package with a public interface + private internals)

```
app/
├── modules/
│   ├── ingestion/       # fetch, parse (Docling), chunk,
│   ├── documents/       # source-agnostic document model
│   ├── retrieval/       # hybrid search (BM25 + vector),
│   ├── generation/      # prompt building, LLM calls, st
```

```
│   ├── agent/           # LangGraph agentic orchestratio
│   ├── memory/          # per-user personalization, conv
│   └── observability/   # tracing, metrics, cost trackin
├── api/                 # FastAPI REST layer (thin — onl
├── core/                # config, DB sessions, auth, DI
└── clients/             # Gradio UI, Telegram bot (thin
```

Golden rules that make it "production-grade":

- The `api/` layer contains **no business logic** — it validates input, calls a module, serializes output.
- Modules **never import each other's internals** — only their public `service.py` / interface. Enforce with `import-linter` (Python).
- Everything crosses boundaries as **Pydantic models**, never raw dicts.
- Every module is independently testable with the rest mocked.

---

2. Tech Stack — Your Choices Reviewed + Recommendations

Your proposed stack is **genuinely strong and coherent** — this is close to how real RAG systems are built. Below is a validation plus targeted upgrades.

✓ Keep as-is (good production choices)

Component	Your pick	Verdict
Containerization	**Docker + Docker Compose**	✓ Correct
API framework	**FastAPI**	✓ Best-in-class for async Python + auto OpenAPI docs

Relational DB	**PostgreSQL**	✓ Metadata, users, jobs, audit
Search / vector	**OpenSearch**	✓ Does **hybrid** (BM25 + kNN vectors) natively with RRF — exactly what you want
Doc parsing	**Docling**	✓ Excellent for PDF/Word/PPT → structured content
Cache	**Redis**	✓ Response cache, rate limiting, dedup
LLM observability	**Langfuse**	✓ Industry standard for traces + prompt versioning
Agent framework	**LangGraph**	✓ Right tool for controllable agentic flows
Chat client	**Telegram + Gradio**	✓ Great for demo/mobile

🔧 Suggested upgrades / additions

Concern	Add / change	Why
Raw file storage	**MinIO** (S3-compatible)	You're ingesting real PDFs/Word files — you need a **blob store** for the originals. Postgres/OpenSearch store text + vectors, not binaries. MinIO runs in Docker and mirrors AWS S3 API so it's cloud-portable.
Orchestration	Keep **Airflow** for learning; know **Prefect**/**Dagster** exist	Airflow is the industry standard (great résumé signal) but heavy. Prefect is lighter/modern. Start with Airflow; you'll appreciate why teams move to Prefect.

Embeddings	**Jina v3** (your pick) is great; also evaluate **BGE-M3**	BGE-M3 gives you dense + sparse + ColBERT vectors from one model — a strong hybrid alternative. Benchmark both in Week 4.
LLM serving	**Ollama** for dev; add a **hosted-API adapter** (Claude) for quality	Ollama is perfect to learn locally & keep costs at zero. But build your generation module behind an **`LLMProvider` interface** so you can swap in the **Claude API** (`claude-opus-4-8` / `claude-sonnet-4-6`) for production-grade answer quality with one config change. This interface pattern is itself a key learning.
DB migrations	**Alembic**	Never hand-edit schema in production. Versioned, reversible migrations.
Auth	**API keys → JWT/OAuth2**	"Personalized" = multi-user = you need identity. FastAPI has first-class OAuth2 support.
System metrics	**Prometheus + Grafana**	Langfuse = LLM traces; Prometheus/Grafana = latency, throughput, error rates, resource use. Real observability needs both.
Testing/ quality	**pytest, Ruff, mypy, pre-commit**	Non-negotiable for production.
CI/CD	**GitHub Actions**	Lint → test → build image → (optionally) deploy on every push.

> **One consolidation option:** if you want *fewer moving parts*, **pgvector** (Postgres extension) can hold your vectors and you'd drop OpenSearch. But you'd lose OpenSearch's superior BM25. **Recommendation: keep OpenSearch** — hybrid search is a headline

> feature and a great thing to learn. Use Postgres for structured data only.

---

3. Weekly Build Plan (12 weeks, escalating complexity)

Each week ends with a **shippable, demoable increment** and a **concrete skill gained**. Weeks 1–7 track your original plan (re-sequenced for cleaner dependencies); Weeks 8–12 push into genuinely advanced, production territory.

Phase 1 — Foundation & Data (Weeks 1–2)

Week 1 — Infrastructure & API skeleton

- `docker-compose.yml` with: FastAPI, PostgreSQL, OpenSearch, Redis, MinIO, Airflow.
- FastAPI app with health checks, config via Pydantic Settings, structured logging.
- Alembic migrations; first tables (`documents`, `sources`, `users`).
- Set up the **modular monolith skeleton** from §1 with `import-linter` enforcing boundaries.
- 🎓 *Skills:* Docker networking, FastAPI project structure, DB schema design, dependency injection.

Week 2 — Source-agnostic ingestion pipeline

- **This is your key differentiator:** not just arXiv. Build a `DocumentSource` abstraction with adapters: `ArxivSource`, `PDFUpload`, `WordUpload`, `MarkdownSource`, `HTMLSource`.
- Store raw files in **MinIO**, parse with **Docling** → normalized `Document` + `Section` model.
- Airflow DAG: daily arXiv sync + an on-demand "ingest uploaded file" flow.
- `POST /documents` endpoint to upload any file type.

- 🎓 *Skills:* abstraction/interface design, ETL orchestration, blob storage, file parsing.

Phase 2 — Retrieval (Weeks 3–4)

Week 3 — Keyword (BM25) search + filtering

- Index parsed docs into OpenSearch with a well-designed mapping.
- BM25 search with metadata filters (source type, date, author, tags) and relevance scoring.
- `POST /search` endpoint with pagination + filters.
- 🎓 *Skills:* inverted indexes, analyzers/tokenizers, relevance tuning, query DSL.

Week 4 — Chunking + hybrid search (BM25 + vectors + RRF)

- Smart chunking (structure-aware: respect headings/paragraphs; overlap; token limits).
- Generate embeddings (**Jina v3** vs **BGE-M3** — benchmark recall).
- kNN vector index in OpenSearch; **Reciprocal Rank Fusion** to combine BM25 + vector.
- Add a **reranker** (e.g., cross-encoder / Jina reranker) as an optional stage.
- 🎓 *Skills:* embeddings, semantic vs lexical retrieval, RRF, reranking, eval.

Phase 3 — Generation & Interface (Week 5)

Week 5 — Full RAG pipeline + streaming + Gradio

- Context builder (top-K chunks, dedup, token-budget packing).
- Prompt template with **inline citations + source metadata**.
- LLM behind an **`LLMProvider` interface** (Ollama now, Claude-swappable later).
- `POST /ask` (sync) and `POST /ask/stream` (SSE streaming).
- Gradio UI: question → streamed answer + sources.
- 🎓 *Skills:* prompt engineering, grounding/citations, SSE streaming,

RAG assembly.

Phase 4 — Production Hardening (Week 6)

Week 6 — Observability, caching, evaluation

- **Langfuse** tracing on every LLM/retrieval step; prompt versioning.
- **Redis** semantic + exact response caching; rate limiting.
- **Prometheus + Grafana** dashboards (latency, cost, error rate, cache hit-rate).
- **RAG evaluation harness** (RAGAS-style): faithfulness, answer relevance, context precision/recall on a gold Q&A set.
- 🎓 *Skills:* distributed tracing, caching strategies, cost control, **eval-driven development**.

Phase 5 — Agentic & Multi-Channel (Week 7)

Week 7 — Agentic RAG (LangGraph) + Telegram

- LangGraph agent: **plan** → **decide tools** → **retrieve** → **(self-)critique** → **answer**.
- Tools: hybrid search, metadata filter, "fetch full paper", "summarize", web search (optional).
- Query routing (simple lookup vs multi-hop reasoning) and query rewriting.
- Telegram bot as a thin client over `/ask-agentic`.
- 🎓 *Skills:* agent orchestration, tool use, state machines, multi-channel design.

Phase 6 — Advanced Intelligence (Weeks 8–10)

Week 8 — Personalization & memory

- Per-user profile: interests, followed topics/authors, reading history.
- Conversation memory (short-term) + long-term preference store (Postgres + vectors).
- Personalized ranking: boost results matching a user's profile.

- Proactive digests: "new papers matching your interests" via Airflow + Telegram push.
- 🎓 *Skills:* personalization, memory architectures, recommendation basics.

Week 9 — Advanced retrieval & reasoning

- **Multi-hop** / **iterative retrieval** (retrieve → reason → retrieve again).
- **HyDE** (hypothetical document embeddings) and **query decomposition**.
- **Contextual/parent-document retrieval** (chunk for search, expand for context).
- Optional **knowledge graph** layer (entities/citations) for "who cites whom" queries.
- 🎓 *Skills:* advanced RAG patterns, agentic retrieval loops, graph thinking.

Week 10 — Multimodal & cross-document synthesis

- Extract & index **tables and figures** from PDFs (Docling supports structure).
- Answer over tables; optional vision model for figure Q&A.
- **Cross-document synthesis:** "compare methods across these 5 papers" with a map-reduce/refine chain.
- Auto-generated literature-review reports.
- 🎓 *Skills:* multimodal RAG, summarization chains, synthesis.

Phase 7 — Production Operations (Weeks 11–12)

Week 11 — Security, auth, CI/CD, testing

- OAuth2/JWT auth; per-user API keys; role-based access; input validation & PII handling.
- **GitHub Actions:** Ruff + mypy + pytest → build image → push to registry.
- Test suite: unit (per module), integration (retrieval+generation), and **RAG eval as a CI gate**.
- Secrets management (`.env` → Docker secrets / Vault);

dependency scanning.
- 🎓 *Skills:* auth/security, automated testing, CI/CD pipelines.

Week 12 — Scale, deploy, document

- Load testing (Locust); async worker queue (Celery/RQ or Airflow) for heavy ingestion.
- Horizontal scale plan; identify which **module** to extract to a service first if needed.
- Deploy to a cloud VM / managed containers; backups for Postgres, OpenSearch, MinIO.
- **Architecture Decision Records (ADRs)**, API docs, runbook, demo video.
- 🎓 *Skills:* load/perf testing, deployment, capacity planning, professional documentation.

---

4. What Makes This "Production-Grade" (checklist)

- ☐ Strict module boundaries enforced in CI (`import-linter`)
- ☐ Everything typed (Pydantic + mypy) and linted (Ruff)
- ☐ Versioned DB migrations (Alembic) — never manual schema edits
- ☐ Config via env/secrets, never hardcoded
- ☐ Auth on every endpoint; rate limiting; input validation
- ☐ Tracing (Langfuse) + metrics (Prometheus/Grafana) + structured logs
- ☐ **Eval suite gates deploys** — you can prove quality didn't regress
- ☐ Caching + graceful degradation (fallback when LLM/search is down)
- ☐ CI/CD: every push is linted, tested, evaluated, built
- ☐ `LLMProvider` / `DocumentSource` interfaces so components are swappable
- ☐ Runbook + ADRs so someone else could operate it

---

5. Skills Map — How This Delivers Your Three Goals

Goal	Where you build it
AI / LLMs	RAG (W4–5), evaluation (W6), agentic reasoning (W7, W9), personalization (W8), multimodal & synthesis (W10)
REST APIs	FastAPI design (W1), async + streaming/SSE (W5), auth/OAuth2 (W11), rate limiting & versioning (W6/W11), OpenAPI docs
Databases	Postgres schema + Alembic (W1), OpenSearch inverted index + kNN (W3–4), Redis caching (W6), MinIO blob store (W2), vector stores & optional graph (W4/W9)

---

6. Suggested First Steps

1. Initialize repo: `git init`, add `pyproject.toml`, Ruff, mypy, pre-commit.
2. Scaffold the modular-monolith folder structure from §1.
3. Write `docker-compose.yml` for Postgres + Redis + OpenSearch + MinIO first (add Airflow once ingestion starts).
4. Stand up FastAPI with a `/health` endpoint and one Alembic migration.
5. Commit early, commit often — treat it like a real product from day one.

---

*Architecture: **Modular Monolith** · Language: **Python 3.11+** · Deploy: **Docker Compose** → **cloud containers**. Start simple, keep boundaries clean, and every week ships something you can demo.*